Si-VISION
INNOVATIVE IC SOLUTIONS

DESIGN DOCUMENTATION FOR

CXS BRIDGE

Encryption & Data Management System

Supervised by :

En.Menna Samir

Prepared by :

Mariam Gomaa Khalaf

Internship Program :

Digital Design Internship

Table of Contents

Table of Figures

1.Introduction:

1.1 Purpose of the Document:

This document presents the design architecture of the CXS Bridge Encryption and Data Management System developed as part of the Si-Vision Digital Design Internship. It provides a comprehensive description of the proposed hardware architecture, including the functionality and interaction of each design block.

The document explains the overall system architecture, the internal organization of each module, and the interfaces between modules. For every functional block, the document describes its purpose, inputs, outputs, internal operation, and role within the complete system.

In addition, this document analyzes the system data flow by illustrating how packets are processed throughout the architecture, from reception at the CXS interface to storage in the Central Data Memory (CDM).

1.2 Design Objectives

Design a modular and scalable architecture for the CXS Bridge Encryption and Data Management System.

Implement a reliable CXS receiver interface capable of handling packet reception, credit-based flow control, and protocol control signal decoding.

Enable efficient packet processing by supporting packet extraction, buffering, encryption, parity generation, and storage.

Ensure proper communication between system modules through well-defined interfaces and control signals.

Support reliable data transfer using the CXS protocol while maintaining protocol compliance.

Provide an architecture that is easy to verify, maintain, and extend with additional features in future implementations.

Optimize the design for efficient RTL implementation by separating the system into independent functional blocks with clear responsibilities.

Analyze the complete packet data flow to verify the correctness of the proposed architecture under different operating scenarios.

2.System Overview

The CXS Bridge Encryption and Data Management System is a hardware architecture designed to receive packets through the CXS interface, process and secure the received data, and store the processed data in the Central Data Memory (CDM). If a packet is detected as invalid or contains an error during processing, an appropriate status is generated and reported to the transmitter software. The architecture consists of several independent functional modules that work together to provide reliable packet handling, flow control, asynchronous buffering, control management, encryption, parity generation, address translation, and memory management.

2.1 System Architecture

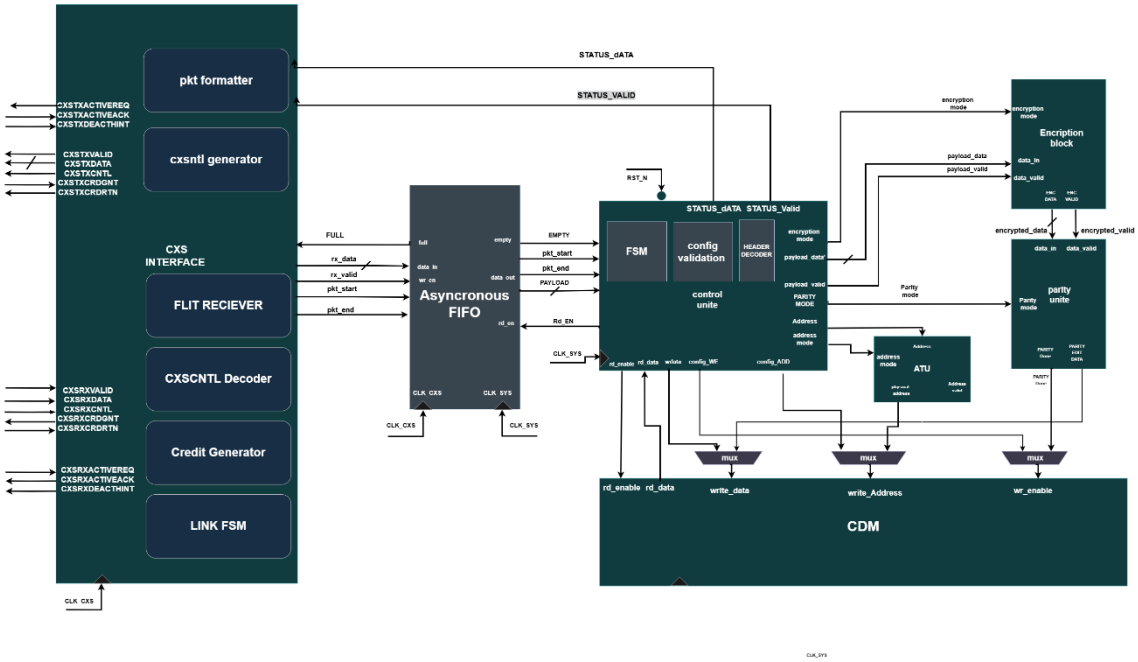

Figure 1 High-Level Architecture of the CXS Bridge Encryption and Data Management System without CDC handling

the proposed high-level architecture of the CXS Bridge Encryption and Data Management System. The system is composed of the CXS Interface, Control Unit, Register File, Encryption Block, Parity Unit, Address Translation Unit (ATU), and Central Data Memory (CDM). These modules cooperate to receive packets from the CXS interface, validate and process packet information, perform encryption and parity generation, translate logical addresses into physical addresses, and store the processed data in memory. The Control Unit coordinates the operation of all functional blocks

2.2 Functional Description

1. **Packet Reception**

   The RX CXS Interface receives packets from the CXS link and manages link activation, credit-based flow control, and validates incoming flits.

   The CXSCNTL field is decoded to identify packet boundaries and protocol information.

2. **Packet Delivery**

   The received packet is converted into the internal 16-bit data format(parametrized).

   The packet data, along with packet control signals (such as start and end of packet), is forwarded to the Control Unit.

3. **Packet Processing**

   The Control Unit interprets the packet header and determines the required operation.

   According to the packet type, it coordinates the subsequent processing stages.

4. **Data Protection**

   The Encryption Block encrypts the payload using the selected encryption mode.

   The Parity Unit generates parity information to support error detection.

5. **Address Translation**

   The Address Translation Unit (ATU) converts the logical address contained in the packet into the appropriate memory address based on the configured translation mode.

6. **Memory Management**

   The processed data is written into the Central Data Memory (CDM).

   The Register File stores configuration parameters, status information, and system control registers used by the Control Unit.

7. **Status Transmission**

   After processing is completed, the Control Unit generates the appropriate status information.

   The TX CXS Interface formats the status into CXS flits and transmits it back through the CXS link using the protocol's flow control mechanism.

2.3 High-Level Data Flow

The system data flow begins when the RX CXS Interface receives a valid flit from the CXS link, performs protocol handling, and decodes the CXSCNTL field. The packet received is forwarded to the Control Unit, which processes the packet header and separates the payload into address and data fields. The data is encrypted and passed through the Parity Unit, while the address is translated by the Address Translation Unit (ATU). Finally, the processed data is stored in the Central Data Memory (CDM), and the generated status is transmitted back through the TX CXS Interface as follows.

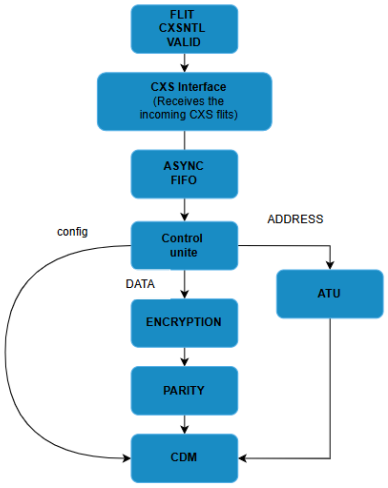

Figure 2 High-Level Data Flow

2.4 Main Functional Blocks

CXS Interface: Receives and transmits CXS packets, performs protocol handling, manages credit-based flow control, and decodes the CXSCNTL field.

Control Unit: Controls the overall system operation, processes packet headers, extracts payload information.

Register File: Stores configuration registers, device information, and status registers used by the Control Unit.

Encryption Block: Encrypts the payload data according to the selected encryption mode.

Parity Unit: Generates parity information for the processed payload to support error detection.

Address Translation Unit (ATU): Translates logical addresses into physical memory addresses based on the selected translation mode.

Central Data Memory (CDM): Stores the encrypted payload together with its associated parity and translated address.

3. Design Architecture

3.1 CXs Interface (Rx)

Purpose

It receives incoming CXS flits from the CXS link, performs protocol handling, manages credit-based flow control, and decodes the CXSCNTL field to determine packet boundaries and control information. The interface buffers the received data and forwards the packet stream, along with the associated control signals, to the Control Unit for further processing.

Inputs & outputs

Signal	Width	Direction	Description
CLK	1	Input	CXS receive clock.
RST_N	1	Input	Active-low reset.
CXSDATA	Configurable (256/512/1024)	Input	Incoming CXS data flit.
CXSCNTL	Configurable 14, 18, 22, 27, 33, 36 or 44 bits. Depends on CXSMAXPKTPERFLIT and CXSDATAFLITWIDTH	Input	CXS control field contains packet boundary.
CXSVALID	1	Input	Indicates that the received flit is valid.
CXSACTIVEREQ	1	Input	Link activation request.
CXSDEACTHINT	1	Input	Indicates that the remote transmitter requests link deactivation.
rx_data	16 (parametrize)	Output	16-bit packet data forwarded to the Control Unit.
rx_valid	1	Output	Indicates that rx_data is valid.
pkt_start	1	Output	Asserted for the first word of a packet.
pkt_end	1	Output	Asserted for the last word of a packet.
packet_error	1	Output	packet contains an error.
CXSCRDGNT	1	Output	Credit granted to the transmitter.
CXSACTIVEACK	1	Output	Acknowledges successful link activation.

Internal Architecture

The RX CXS Interface consists of four main functional blocks that cooperate to receive CXS flits, decode protocol control information, manage flow control, and forward the received packet stream to the Control Unit.

3.1.1. Flit Receiver

The Flit Receiver is the entry point of the RX CXS Interface. It samples the incoming CXSDATA and CXSCNTL signals on the rising edge of the CXS clock when CXSVALID is asserted. The received flit is registered and forwarded for further processing, while the CXSCNTL field is sent to the CXSCNTL Decoder.

Signal	Direction	Description
CXS_CLK	Input	CXS receive clock
RST_N	Input	Active-low reset
CXSDATA	Input	Incoming flit
CXSCNTL	Input	CXS control field
CXSVALID	Input	Valid flit indication
rx_data	Output	Registered flit data
cxscntl_data	Output	Registered control field
Rx_valid	Output	Indicates a valid received flit

3.1.2. CXSCNTL Decoder

The CXSCNTL Decoder interprets the protocol control information contained in the CXSCNTL field. It identifies the start and end of packets, extracts the associated packet pointers, and detects packet continuation or error information. The decoded metadata is forwarded to the FIFO together with the corresponding flit to preserve the packet boundaries during buffering.

Signal	Direction	Description
cxscntl_data	Input	Registered CXSCNTL
flit_valid	Input	Valid flit
start_field	Output	Decoded START field
end_field	Output	Decoded END field
end_error[]	Output	ENDERROR field

3.1.3. Credit Generator

Purpose

The Credit Generator manages the CXS credit-based flow control mechanism by monitoring the available space in the asynchronous FIFO. It grants credits to the remote transmitter whenever sufficient buffer space is available, allowing additional flits to be transmitted. This mechanism prevents FIFO overflow and ensures reliable data reception.

Inputs & Outputs

Signal	Width	Direction	Description
CLK_CXS	1	Input	CXS interface clock.
RST_N	1	Input	Active-low reset.
fifo_full	1	Input	Indicates the FIFO is full.
Read_done	1	Input	Indicates the FIFO has one read operation done .
CXSCRDGNT	1	Output	Credit grant signal sent to the remote transmitter.

Operation

The Credit Generator continuously monitors the available space in the asynchronous FIFO and if there is a successful reading from the fifo , it asserts **CXSCRDGNT** to grant a credit to the remote transmitter, indicating that another flit can be transmitted. As flits are written into the FIFO, the number of available entries decreases. If the FIFO becomes full, the Credit Generator deasserts **CXSCRDGNT**, preventing the transmitter from sending additional flits. Once the Control Unit reads data from the FIFO and frees buffer space, the Credit Generator reasserts **CXSCRDGNT**, allowing transmission to resume.

3.1.4. Link control

Link FSM manages the operational state of the CXS interface. It handles the link activation and deactivation handshake using the CXS control signals and controls the transition between the Reset, Inactive, Active, and Deactivation states. This mechanism enables reliable link management while reducing power consumption when the interface is idle.

Signal	Direction	Description
CLK	Input	Clock of cxs
CXSACTIVEREQ	Input	Activation request
CXSDEACTHINT	Input	Deactivation request
RST_N	Input	RST_N
CXSACTIVEACK	Output	CXSACTIVEACK

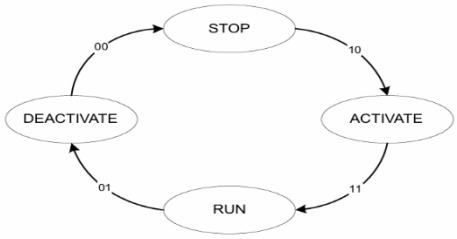

Figure 3 FSM for link control

3.2 Control Unit

Purpose

The Control Unit is the central controller of CXS Bridge. It receives the formatted packet stream from the **RX Interface** and manages the system operation according to the current state of the communication protocol.

The Control Unit identifies the beginning and end of each packet using the **pkt_start and pkt_end** signals. When the first payload word of a packet is received (pkt_start = 1), it is interpreted as the packet header. The Header Decoder extracts the packet type and other control information, allowing the FSM to determine the required operation.

For Configuration Packets, the Control Unit validates the packet contents, stores the configuration parameters in the Register File, and updates the system configuration.

For Link Packets, the Control Unit verifies the packet header and changes the system state to the Link-Up state.

For Data Packets, the Control Unit separates each 16-bit payload word into an 8-bit address and an 8-bit data field.

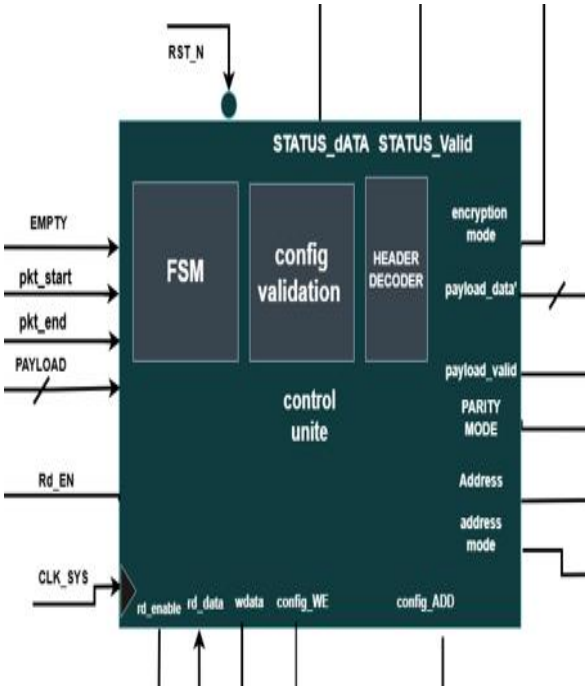

Figure 4 control unit block diagram and itis internal logic

If any validation error is detected during packet processing, the Control Unit generates an appropriate status packet that is transmitted back to the remote transmitter

Inputs & outputs

Signal	Width	Direction	Description
CLK_SYS	1	Input	System clock.
RST_N	1	Input	Active-low reset.
payload	16	Input	Packet data received from the RX Interface.
empty	1	Input	Indicates that fifo is empty.
pkt_start	1	Input	Indicates the start of a packet.
pkt_end	1	Input	Indicates the end of a packet.
rd_data	16	Input	Data read from the cdm.
cfg_we	1	Output	Enables writing configuration registers.
rf_addr	parametrize	Output	Configuration address in cdm.
rf_data	16	Output	Configuration data.
payload_data	8	Output	Payload data to the Encryption Block.
payload_addr	8	Output	Payload address to the ATU.
payload_valid	1	Output	Indicates valid payload data.
status_data	parametrize	Output	Status packet sent to the TX Interface.
status_valid	1	Output	Indicates valid status information.
Party_mode	1	Output	Odd or even parity
Address_mode	1	Output	Region or Limit Address and Last Address Mode.
Encryption_mode	1	Output	Even or odd

Rd_en	1	Output	For enable reading from FIFO
Op_type	1	Output	Choose the operation on CDM

Internal logic

Finite State Machine (FSM)

Header Decoder

Configuration Validation

The Control Unit is implemented as a Finite State Machine (FSM) responsible for controlling the receiver operation throughout the different stages of packet processing. The FSM manages configuration packet decoding, link initialization, payload reception, and error handling by generating the appropriate control signals for each operating state. Figure X.X illustrates the state transition diagram of the Control Unit FSM.

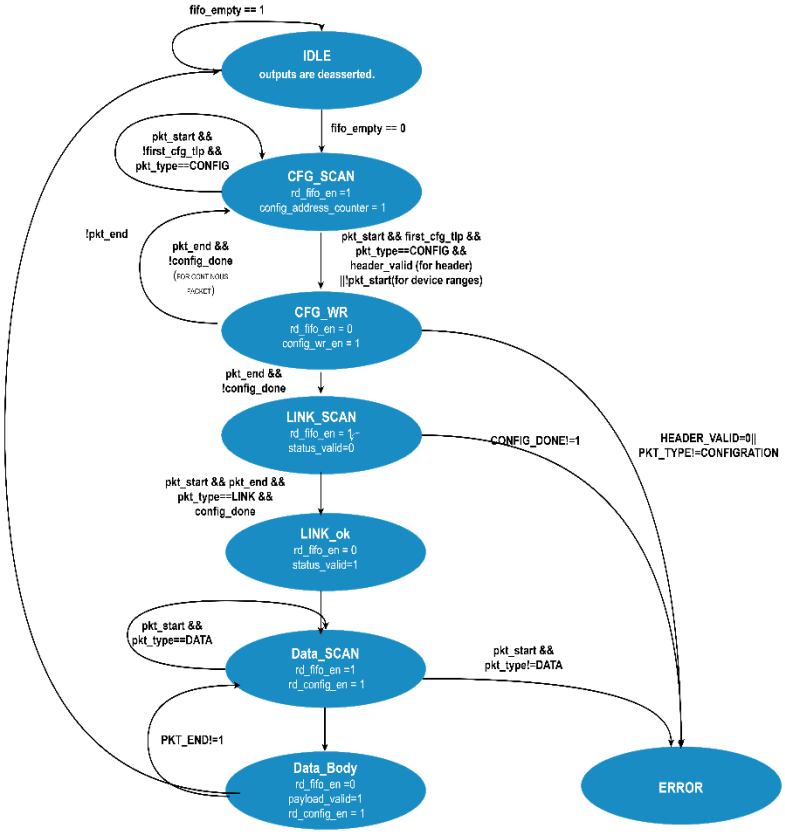

Figure 5 Finite state machine for operation control

IDLE State

The **IDLE** state is the default state after reset. In this state, all control outputs are deasserted, and the receiver remains inactive while waiting for incoming data in the FIFO. If the FIFO is empty (fifo_empty = 1), the FSM remains in the IDLE state. Once data becomes available (fifo_empty = 0), the FSM transitions to the **CFG_SCAN** state to begin packet processing.

CFG_SCAN State

The **CFG_SCAN** state is responsible for reading the first flit of an incoming packet and determining whether it is a configuration packet. During this state, FIFO reading is enabled and the configuration address counter is initialized to prepare for configuration decoding.

If the received packet is identified as the first configuration packet (pkt_start, first_cfg_tp, and pkt_type = CONFIG), the FSM transitions to the **CFG_WR** state. Otherwise, if additional configuration flits are required before completing the configuration process (pkt_end and !config_done), the FSM remains in the CFG_SCAN state until all configuration information has been received.

CFG_WR State

The **CFG_WR** state writes the decoded configuration information into the internal configuration registers. During this state, configuration writing is enabled while FIFO reading is temporarily disabled.

LINK_SCAN State

The **LINK_SCAN** state verifies the received link configuration and ensures that all required configuration parameters have been correctly received before normal data transfer begins. During this state, FIFO reading is enabled while the link status remains invalid.

If the configuration process completes successfully (config_done = 1), the FSM transitions to the **LINK_OK** state. Otherwise, if an unexpected packet type or invalid configuration is detected, the FSM enters the **ERROR** state.

LINK_OK State

The **LINK_OK** state indicates that the receiver has completed the link initialization process successfully and is ready to receive normal data packets. In this state, the link status output is asserted to indicate that the receiver is operating normally.

DATA_SCAN State

The **DATA_SCAN** state identifies the beginning of a payload packet and prepares the receiver for payload extraction. During this state, FIFO reading and configuration reading are enabled to obtain the required information for payload processing.

Once a valid data packet has been detected, the FSM transitions to the **DATA_BODY** state to receive the packet payload.

DATA_BODY State

The **DATA_BODY** state is responsible for receiving and processing the payload data. During this state, the payload valid signal is asserted to indicate that the output payload data is valid, while the stored configuration information remains accessible.

The FSM remains in this state until the end of the current packet is detected (PKT_END = 1). After the packet has been completely received, the FSM returns to the **IDLE** state to wait for the next incoming packet.

ERROR State

The **ERROR** state is entered whenever the receiver detects an invalid packet sequence or a protocol violation. Examples include receiving an unexpected packet type, an invalid configuration header, or an incorrect transition during the initialization sequence.

Table of states

State	Outputs Asserted	Description
IDLE	None	Initial state of the FSM. The controller waits until the FIFO contains a packet. No read, write, or status signals are asserted.
CFG_SCAN	`rd_fifo_en = 1` `config_address_counter = 1`	Reads one flit from the FIFO and determines whether it is the first configuration header, a continuation configuration header, or a configuration payload. The configuration address counter is enabled to prepare for storing configuration data.
CFG_WR	`config_we = 1`	Writes the current configuration payload flit into the Configuration Data Memory (CDM). The FSM checks whether the current packet or the entire configuration phase has finished before determining the next state.
LINK_SCAN	`rd_fifo_en = 1`	Reads the Link-Up packet from the FIFO and verifies that it is a valid Link packet received after all configuration packets have been stored.
LINK_OK	`status_valid = 1`	Indicates that the Link-Up packet has been successfully validated and the system is ready to start data transmission. This state lasts for one clock cycle before entering the data phase.
DATA_SCAN	`rd_fifo_en = 1` `rd_config_en = 1`	Reads one flit from the FIFO and simultaneously requests the corresponding configuration data from the CDM. The packet header is verified and skipped before processing payload flits.
DATA_BODY	`payload_valid = 1` `rd_config_en = 1`	Processes one payload flit of the data packet. The payload is marked as valid while the previously read configuration data is available for the encryption or processing stage.
ERROR	`status_valid = 1`	Entered whenever an invalid header, incorrect packet type, or protocol violation is detected. The status signal indicates the error condition, and the FSM remains in this state until reset.

3.3 Encryption Module

Purpose

The **Encryption Block** is a combinational logic block responsible for encrypting the payload data received from the Control Unit according to the configured encryption mode. Upon receiving a valid payload, the block immediately performs the encryption without storing intermediate data or requiring additional clock cycles.

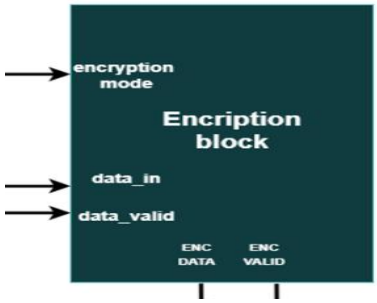

Figure 6 the block diagram for Encryption unit

Inputs/Outputs

Signal	Width	Direction	Description
payload_data	8	Input	Payload data received from the Control Unit.
payload_valid	1	Input	Indicates that the input payload is valid.
encryption_mode	1	Input	Select the encryption mode stored in the Register File.
encrypted_data	16	Output	Encrypted payload after bit expansion.
encrypted_valid	1	Output	Indicates that the encrypted output is valid.

Encryption Flow

The Encryption Block starts processing when payload_valid is asserted at three stages as follows:
First counts the number of logic '0' and logic '1' bits in the input payload to determine whether the special encryption condition applies.

Second check the encryption mode signal to choose from two encryption modes as follows:
Even Mode
Each input bit is expanded by appending 0 example for mapping:
0 → 00
1 → 10
Odd Mode
Each input bit is expanded by appending 1 example for mapping:
0 → 01
1 → 11
Special Case
If the number of logic '0' bits equals the number of logic '1' bits, or if both counts are even, the block overrides the selected mode and applies the special mapping:
0 → 01
1 → 10

3.4 Parity Module

Purpose

The Parity Block is a combinational logic block responsible for generating a parity bit for the encrypted payload received from the Encryption Block. The parity bit is generated according to the parity mode stored in the Register File, which specifies either Even Parity or Odd Parity. When a valid encrypted payload is received, the block immediately computes the parity bit and appends it to the encrypted data before forwarding the result to the Central Data Memory (CDM).

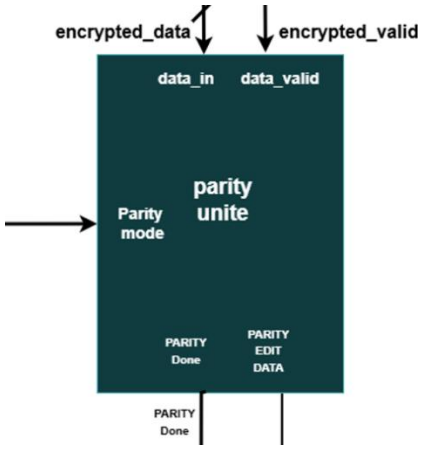

Figure 7 the block diagram for parity unit

Inputs & Outputs

Signal	Width	Direction	Description
encrypted_data	16	Input	Encrypted payload from encryption block.
encrypted_valid	1	Input	Indicates that the encrypted payload is valid.
parity_mode	1	Input	Selects the parity generation mode (Even or Odd).
data_with_parity	17	Output	Encrypted payload with the appended parity bit.
parity_done	1	Output	Indicates that the output data and parity bit are valid.

3.5 Central Data Memory

Purpose

The Central Data Memory (CDM) is the main storage unit of the CXS Bridge. It stores both the system configuration and the payload data received from the CXS interface. The CDM is divided into memory regions assigned to different devices, ensuring that each device accesses only its allocated address space. A local CDM Controller manages memory writes and verifies that incoming data is written to the correct device region according to the stored configuration.The memory width and depth are parameterized, allowing the CDM size to be configured according to system requirements.

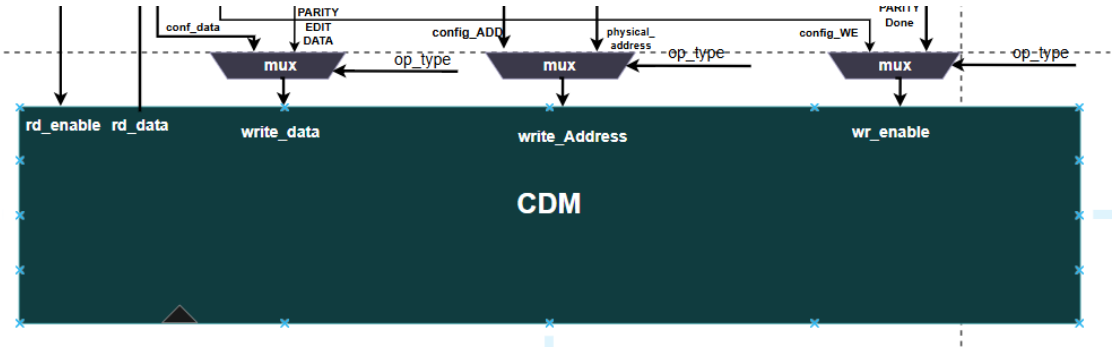

Figure 8 the block diagram for CDM

Memory organization

The CDM consists of two logical regions:

- **Configuration Region:** Stores the configuration packet beginning at address 0x0000, including the encryption mode, parity mode, number of devices, and device memory-region information.
- **Payload Region:** Stores payload data belonging to each configured device. The physical address is generated by the ATU based on the logical address contained in the data packet.

CDM Controller

The CDM includes a local controller responsible for managing memory operations.

Its main functions are:

- Receive written requests from the Control Unit.
- Verify that the destination address belongs to the configured device region.
- Allow writing only to valid memory locations.
- Generate a write-complete or error status.

Input & Output

Signal	Width	Direction	Description
CLK_SYS	1	Input	System clock.
RST_N	1	Input	Active-low reset.
WR_Address	8 (Parameterized)	Input	Physical memory address generated by the Address Translation Unit (ATU).
addr_valid	1	Input	Indicates that the translated address is valid.
WData	Parameterized	Input	Encrypted payload with the generated parity bit received from the Parity Block.
data_valid	1	Input	Indicates that the input data is valid from parity block.
WR_EN	1	Input	Write enable generated by the Control Unit.
RD_EN	1	Input	Read enable generated by the Application.
RD_Address	8 (Parameterized)	Input	Physical memory address generated by Application.
RD_DATA	Parameterized	Output	Data read by internal logic blocks (Application).

3.6 Asynchronous FIFO

Purpose

The Asynchronous FIFO provides a buffer between the CXS Interface and the Control Unit, enabling safe data transfer between the **CXS clock domain (CLK_CXS)** and the **System clock domain (CLK_SYS)**. It temporarily stores the received payload words and their associated packet boundary information until the Control Unit is ready to process them. The FIFO also supports the credit-based flow control mechanism by indicating the available receive buffer space.

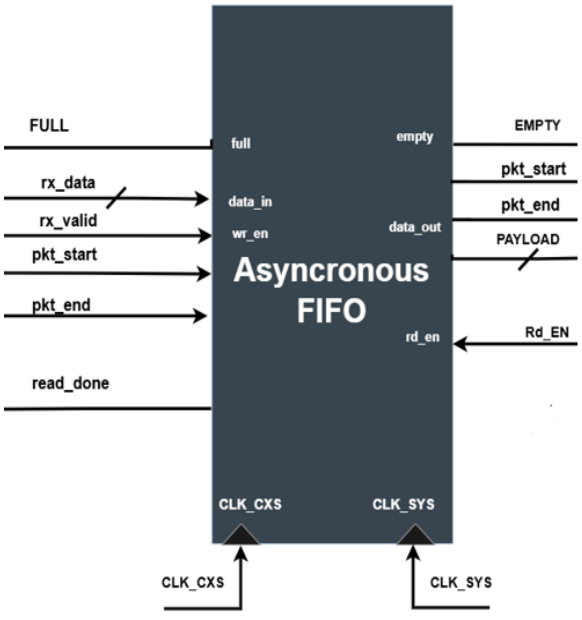

Figure 9 Asynchronous FIFO block diagram

Internal FIFO Format

Each FIFO entry stores one **16-bit payload word** together with two control bits that indicate the packet boundaries. The control bits are generated by the **CXSCNTL Decoder** in the RX Interface and stored with the corresponding payload. This allows the Control Unit to identify the beginning and end of each packet without reprocessing the CXS control information.

Each FIFO entry has the following format:

Field	Width	Description
pkt_end	1 bit	Indicates that this payload word is the last word of the packet.
pkt_start	1 bit	Indicates that this payload word is the first word of the packet.
payload	16 bits	Payload word extracted from the received CXS flit.

The total FIFO entry width is **18 bits**.

Input & Output

Signal	Width	Direction	Description
wr_clk	1	Input	CXS interface clock (CLK_CXS).
rd_clk	1	Input	System clock (CLK_SYS).
rst_n	1	Input	Active-low reset.
wr_en	1	Input	Write enable from the RX Interface.
rd_en	1	Input	Read enable from the Control Unit.
payload_in	16	Input	16-bit payload received from the RX Interface.
pkt_start_in	1	Input	Indicates the first payload word of a packet.
pkt_end_in	1	Input	Indicates the last payload word of a packet.
payload_out	16	Output	16-bit payload sent to the Control Unit.
pkt_start_out	1	Output	Indicates the first payload word of the packet.
pkt_end_out	1	Output	Indicates the last payload word of the packet.
full	1	Output	FIFO full indicator.
empty	1	Output	FIFO empty indicator.